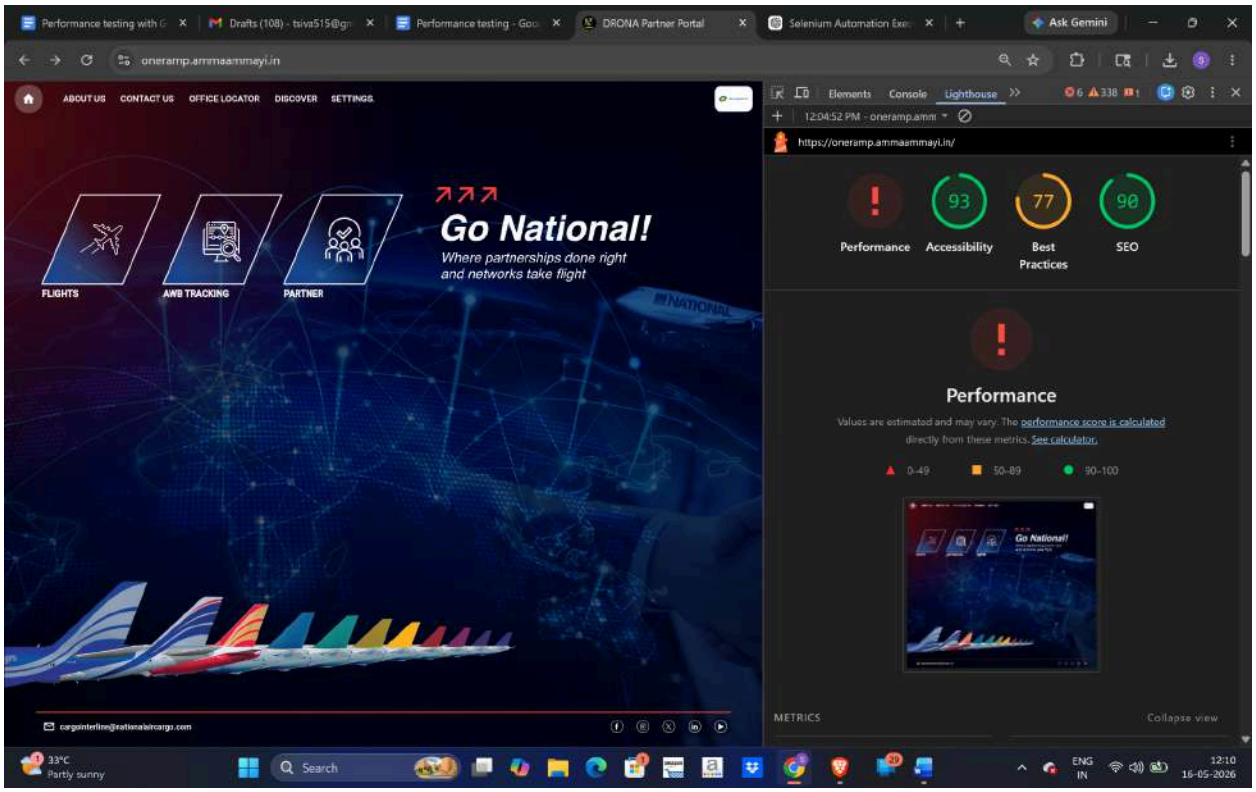
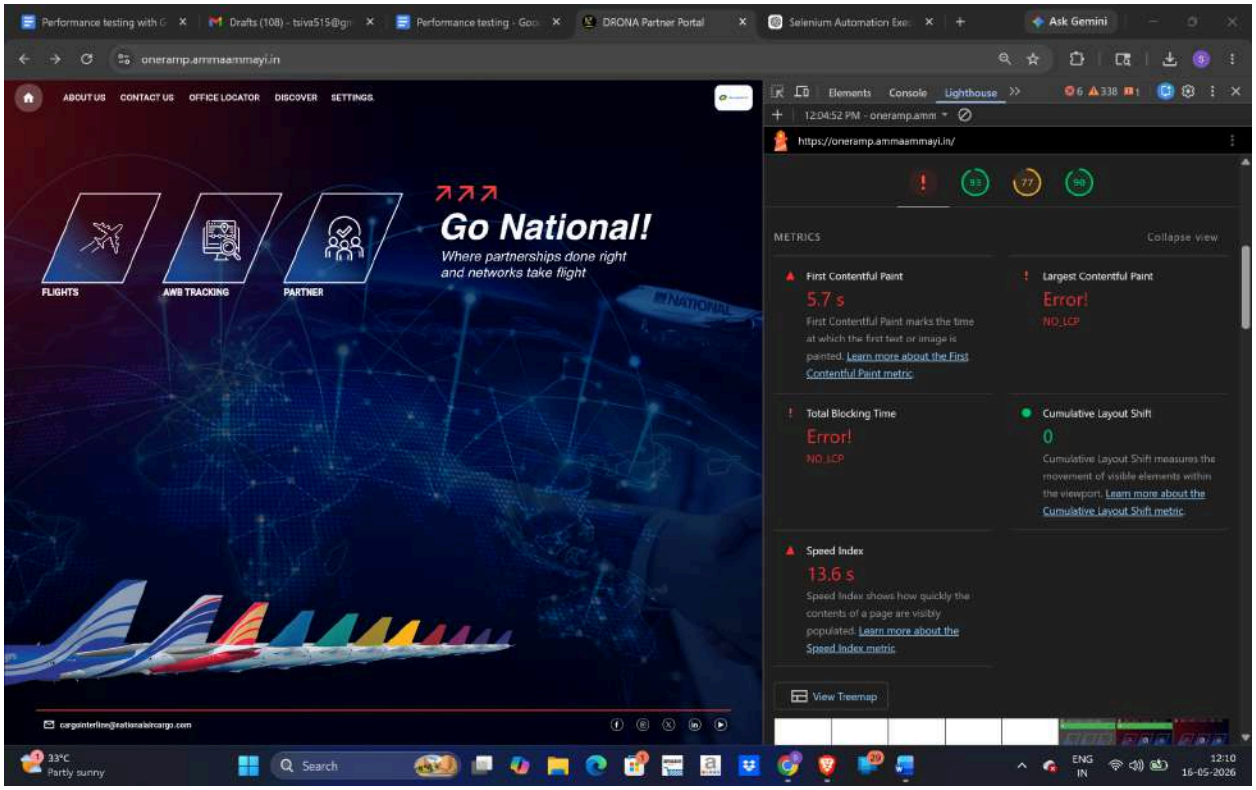
Performance Testing

- Test page load time.
- Test API response time.
- Test checkout flow under multiple users.
- Identify bottlenecks.

1.Test page load time

Performance Testing – Page Load Time





Application Tested:
https://oneramp.ammaammayi.in/

Module:
Customer Portal Landing Page

Tool Used:
Google Lighthouse (Chrome DevTools)

Objective:
To verify frontend page loading performance, rendering behavior, and UI responsiveness.

Test Procedure:
1. Opened application in Google Chrome browser.
2. Opened Chrome DevTools using F12.
3. Navigated to Lighthouse tab.
4. Selected Performance audit.
5. Executed page performance analysis.
6. Captured Lighthouse metrics and diagnostics.

Observed Metrics:

- Accessibility Score: 93

- Best Practices Score: 77
- SEO Score: 90
- First Contentful Paint (FCP): 5.7 seconds
- Speed Index: 13.6 seconds
- Cumulative Layout Shift (CLS): 0

Performance Observations:
- Initial page rendering was delayed due to heavy UI graphics and background assets.
- Large visual content increased page load duration.
- Multiple render-blocking requests affected frontend performance.
- Legacy JavaScript and unused CSS/JavaScript resources impacted optimization efficiency.
- Large network payload size increased loading time.
- UI remained visually stable during rendering because CLS value was 0.

Diagnostics Identified:
- Render-blocking requests
- Legacy JavaScript usage
- Unused CSS
- Unused JavaScript
- Large network payload
- Main-thread execution delay

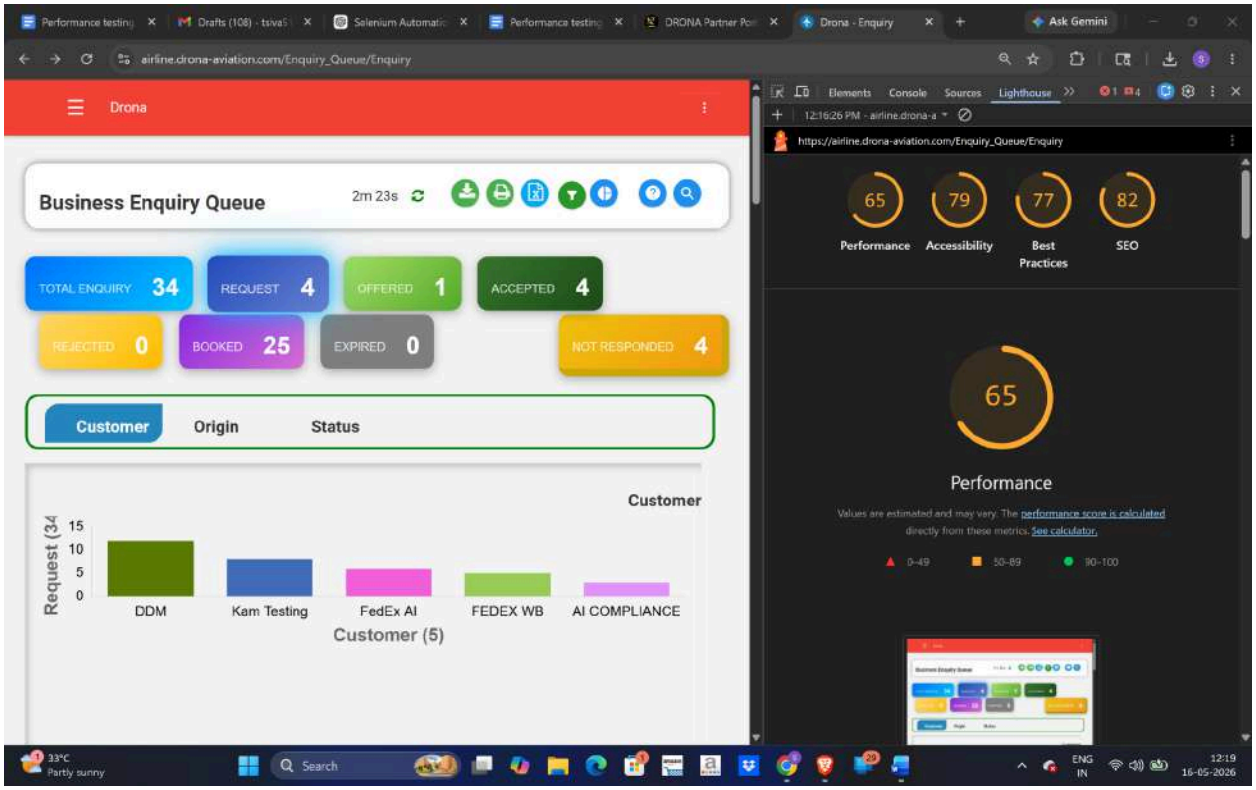
Conclusion:
Application functionality is stable, but frontend optimization is recommended to improve initial load speed and rendering performance.

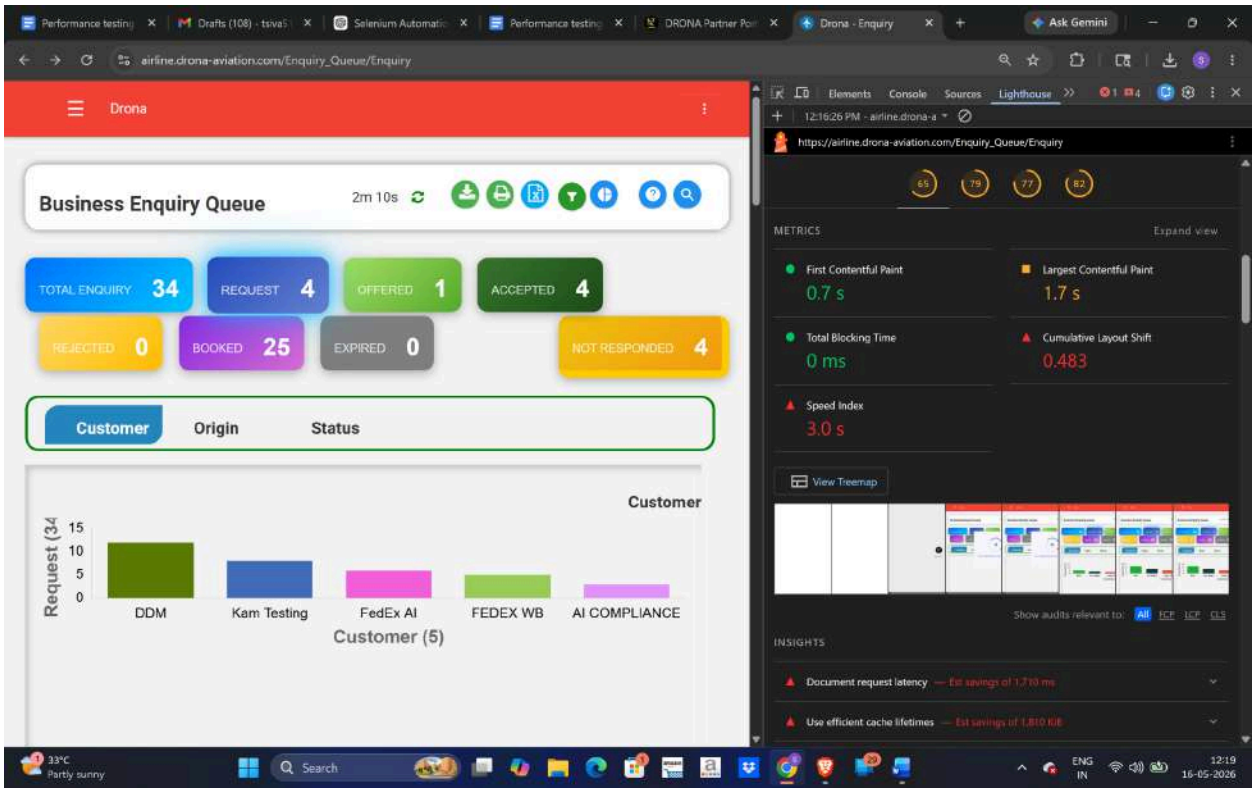
Risk Level:
Medium

WHY PERFORMANCE LOW?

Performance reduction mainly caused by:
- Heavy background graphics
- Large media assets
- Render-blocking scripts
- Unoptimized frontend resources

Performance Testing – Airline Portal Dashboard





Application URL:
https://airline.drona-aviation.com/Enquiry_Queue/Enquiry

Module Tested:
Business Enquiry Queue Dashboard

Tool Used:
Google Lighthouse (Chrome DevTools)

Objective:
To evaluate page load performance, rendering speed, frontend responsiveness, and dashboard optimization of the Airline Portal.

Test Procedure:
1. Opened Airline Portal dashboard in Google Chrome.
2. Opened Chrome DevTools using F12.
3. Navigated to Lighthouse tab.
4. Selected Performance, Accessibility, Best Practices, and SEO audits.
5. Executed Lighthouse page analysis.
6. Captured metrics, diagnostics, and optimization insights.

Observed Metrics:
- Performance Score: 65
- Accessibility Score: 79
- Best Practices Score: 77
- SEO Score: 82

Performance Metrics:
- First Contentful Paint (FCP): 0.7 sec
- Largest Contentful Paint (LCP): 1.7 sec
- Total Blocking Time (TBT): 0 ms
- Speed Index: 3.0 sec
- Cumulative Layout Shift (CLS): 0.483

Observations:
- Dashboard loaded successfully without crash or freeze.
- Initial content rendering was fast.
- Dynamic enquiry widgets and charts affected layout stability.
- CLS value was high due to UI shifting during dashboard rendering.
- Multiple frontend assets increased rendering complexity.
- Dashboard counters, enquiry cards, and chart components affected frontend performance.

Performance Insights Identified:
- Render-blocking requests detected.
- Legacy JavaScript impacted optimization.
- Layout shift culprits identified during rendering.

- Large frontend asset delivery increased payload size.
- Network dependency chain increased request complexity.
- Unused JavaScript and CSS resources affected performance efficiency.

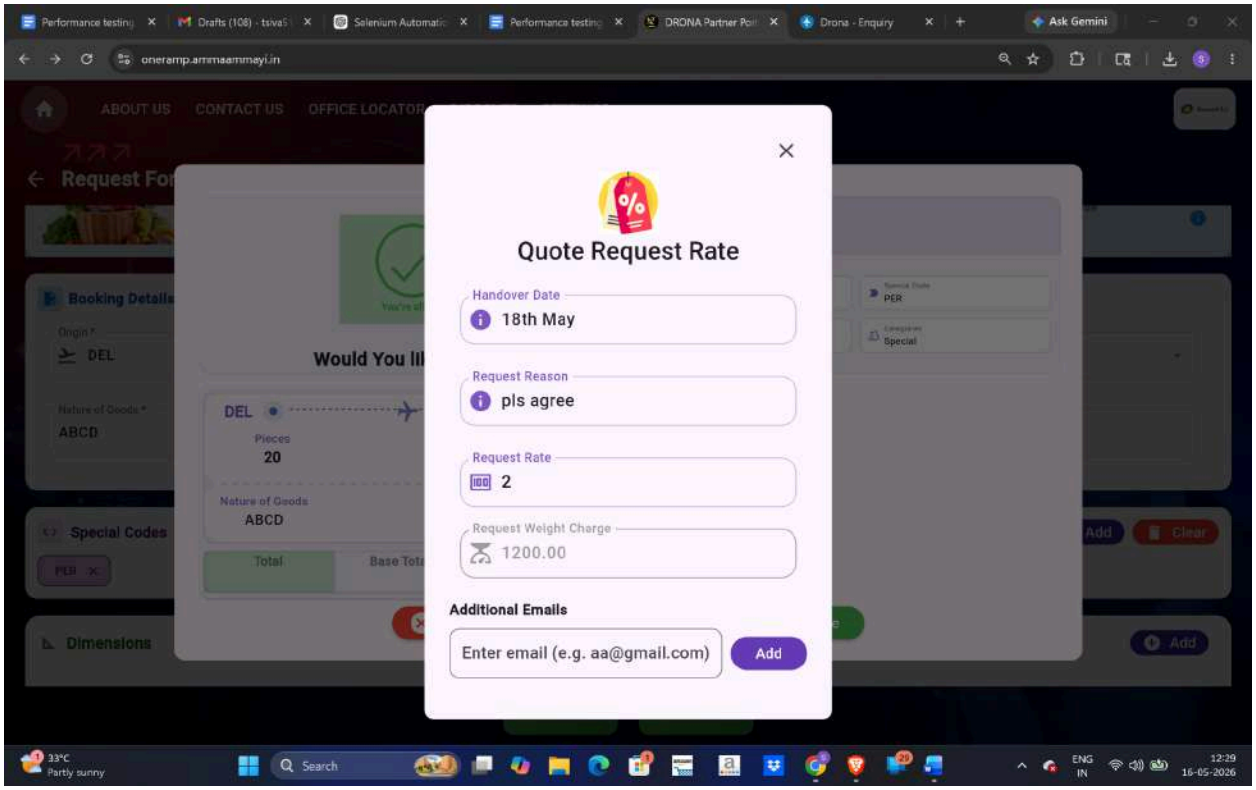
Diagnostics:
- Reduce unused JavaScript
- Minify JavaScript
- Minify CSS
- Improve image delivery
- Optimize font display
- Reduce render-blocking resources

Bottlenecks Identified:
1. Dynamic dashboard chart rendering
2. Multiple enquiry status widgets
3. Layout shifting during page load
4. Large frontend resource payloads
5. Legacy JavaScript dependencies

Conclusion:
The Airline Dashboard is functionally stable and responsive for business operations. However, frontend optimization is recommended to improve rendering stability, reduce layout shifting, and enhance overall dashboard performance.

Risk Level:
Medium

RFQ V2 Module Performance Testing



Performance Testing – RFQ V2 Workflow

Application URL:
https://oneramp.ammaammayi.in/

Module Tested:
Request For Quote (RFQ) V2 Workflow

Tool Used:
Google Lighthouse (Chrome DevTools)

Objective:
To evaluate frontend rendering performance, popup responsiveness, and workflow stability during RFQ booking and quote request operations.

Test Procedure:
1. Logged into Customer Portal.
2. Opened RFQ V2 workflow.
3. Selected product and entered booking details.
4. Opened verification and quote request popup.
5. Executed Lighthouse performance audit.
6. Captured metrics and diagnostics.

Observed Metrics:
- Accessibility Score: 93
- Best Practices Score: 77
- SEO Score: 90
- First Contentful Paint (FCP): 5.1 sec
- Speed Index: 13.9 sec
- Cumulative Layout Shift (CLS): 0

Additional Observations:
- Largest Contentful Paint (LCP) returned NO_LCP error during Lighthouse reload analysis.
- Total Blocking Time (TBT) also returned NO_LCP error because the workflow redirected to landing page during audit reload.

Performance Observations:
- RFQ workflow rendering involved dynamic forms, validation popups, and quote request components.
- Initial rendering delay observed due to large frontend assets and workflow components.
- Multiple render-blocking requests affected frontend loading speed.
- Heavy JavaScript and CSS processing impacted rendering performance.
- UI layout remained visually stable because CLS value remained 0.
- During Lighthouse reload execution, application redirected to landing page due to workflow/session refresh behavior.

Insights Identified:
- Use efficient cache lifetimes
- Render-blocking requests
- Legacy JavaScript usage
- Large frontend payload size
- Third-party dependency loading

Diagnostics Identified:
- Minify CSS
- Minify JavaScript
- Reduce unused CSS
- Reduce unused JavaScript
- Avoid long main-thread tasks
- Large network payload size
- Cache restoration issue during reload

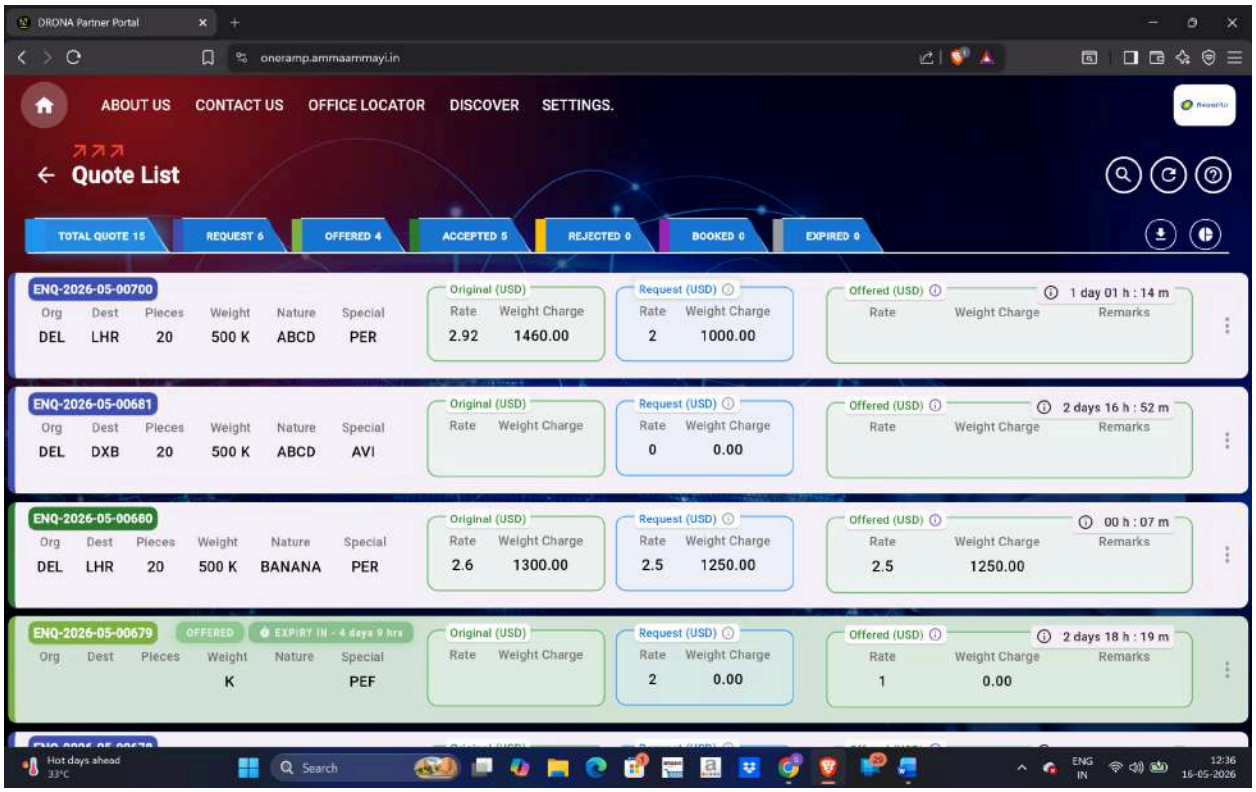
Bottlenecks Identified:
1. Dynamic RFQ form rendering
2. Popup validation components
3. Heavy frontend JavaScript processing
4. Large CSS and asset payloads

5. Session refresh behavior during Lighthouse reload

Conclusion:
The RFQ V2 workflow is functionally stable and responsive during booking operations. However, frontend optimization and session-state handling improvements are recommended to improve rendering performance and reduce frontend processing overhead.

Risk Level:
Medium

Quote List Module Performance Testing



Performance Testing – Quote List Workflow

Application URL:
https://oneramp.ammaammayi.in/

Module Tested:
Quote List Bargaining Workflow

Tool Used:

Google Lighthouse (Chrome DevTools)

Objective:
To evaluate rendering performance, UI responsiveness, and frontend behavior during quote bargaining and enquiry management workflow.

Test Procedure:
1. Logged into Customer Portal.
2. Opened Quote List workflow.
3. Executed Lighthouse performance audit.
4. Captured metrics, insights, and diagnostics.

Observed Metrics:
- Accessibility Score: 93
- Best Practices Score: 77
- SEO Score: 90
- First Contentful Paint (FCP): 5.2 sec
- Speed Index: 9.0 sec
- Cumulative Layout Shift (CLS): 0

Additional Observations:
- Largest Contentful Paint (LCP) returned NO_LCP error during Lighthouse reload analysis.
- Total Blocking Time (TBT) also returned NO_LCP error because workflow redirected during page reload analysis.

Performance Observations:
- Multiple enquiry cards and bargaining components increased frontend rendering complexity.
- Status refresh handling and action buttons affected rendering responsiveness.
- Dynamic enquiry rendering caused moderate frontend processing delay.
- Large frontend payload and JavaScript processing impacted loading efficiency.
- UI layout remained stable during rendering because CLS value remained 0.

Insights Identified:
- Use efficient cache lifetimes
- Render-blocking requests
- Legacy JavaScript usage
- Third-party dependency loading

Diagnostics Identified:
- Minify CSS
- Minify JavaScript
- Reduce unused CSS
- Reduce unused JavaScript
- Avoid long main-thread tasks

- Cache restoration issue during reload analysis

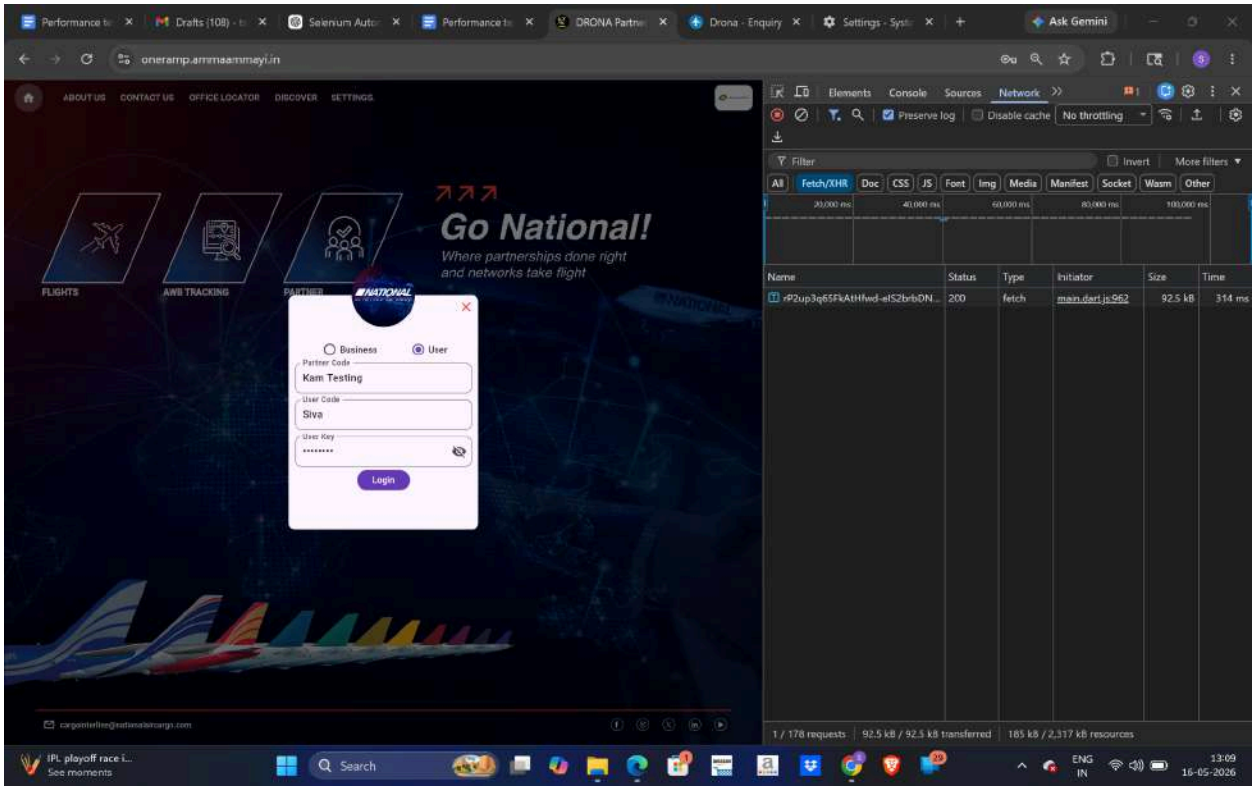
Bottlenecks Identified:
1. Dynamic enquiry card rendering
2. Bargaining workflow components
3. Frontend JavaScript processing
4. Large CSS and frontend payload size
5. Session refresh behavior during reload

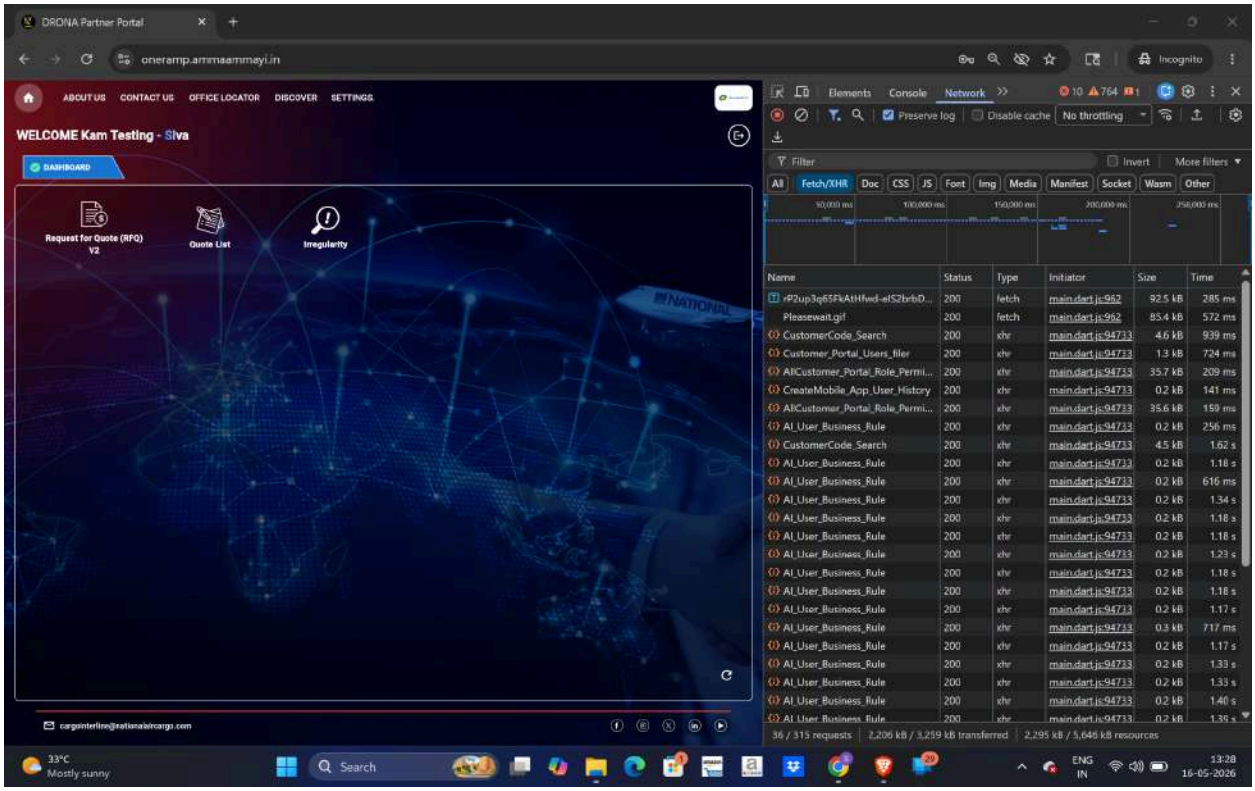
Conclusion:
The Quote List workflow handled bargaining operations successfully and remained functionally stable during testing. However, frontend optimization is recommended to improve rendering efficiency and reduce processing overhead during enquiry workflow operations.

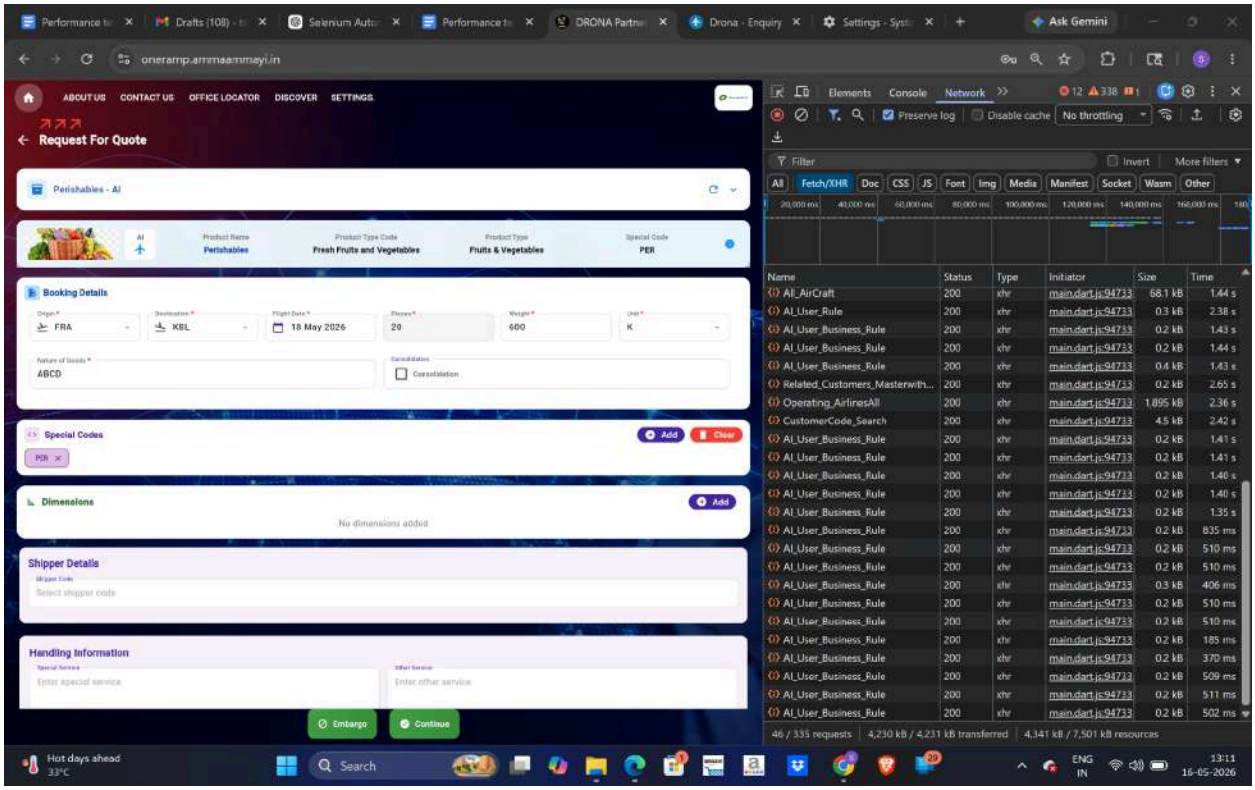
Risk Level:
Medium

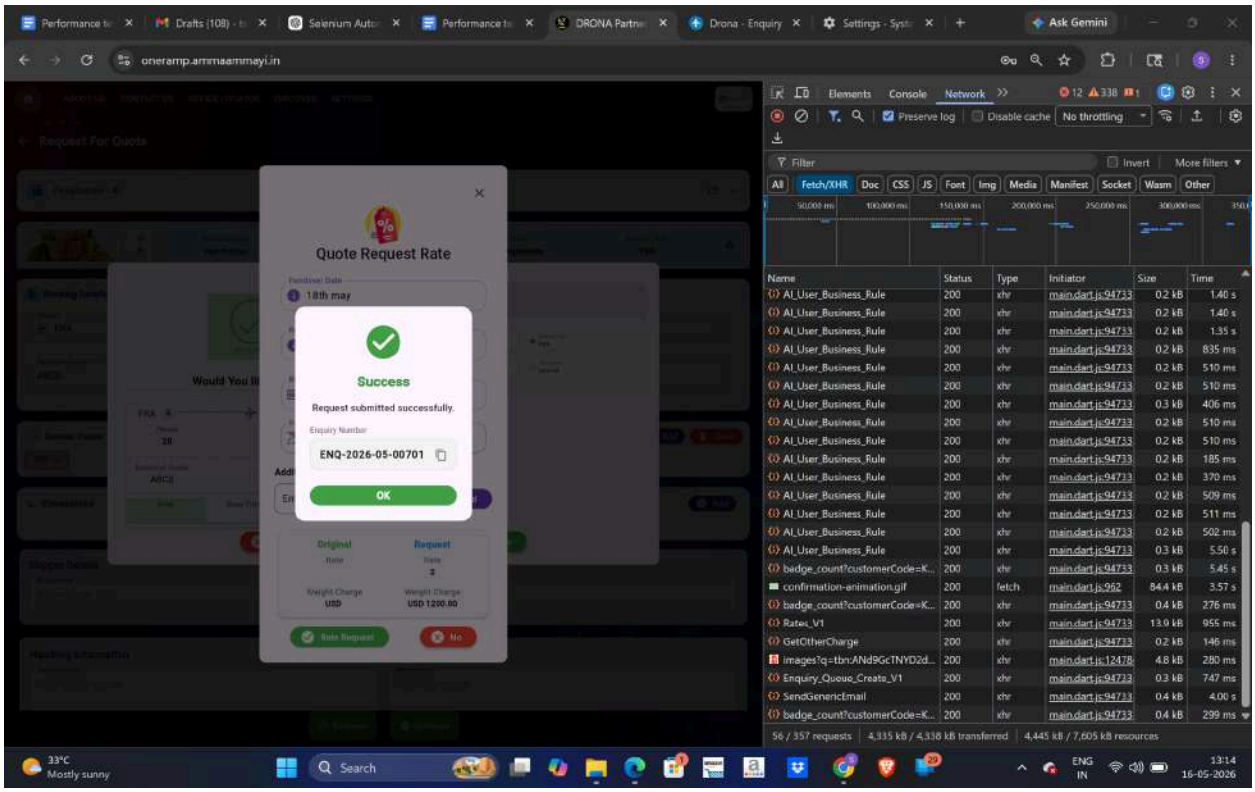
2.API Response Time Testing

Using: chrome DebTools


oneramp.arrmaammayi.in
ABOUT US
CONTACT US
OFFICE LOCATOR
DISCOVER
SETTINGS
FLIGHTS
AWB TRACKING
Go National!
Where partnerships done right and networks take flight
Business
User
Partner Code
Kam Testing
User Code
Siva
User Key
Login
Elements
Console
Sources
Network
Preserve log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
Name
Status
Type
Initiator
Size
Time
1 / 178 requests
92.5 kB / 92.5 kB transferred
185 kB / 2,317 kB resources


DRONA Partner Portal
oneramp.arrmaammayi.in
ABOUT US
CONTACT US
OFFICE LOCATOR
DISCOVER
SETTINGS
WELCOME Kam Testing - Siva
DASHBOARD
Request for Quote (RFQ) V2
Quote List
Irregularity
Elements
Console
Network
Preserve log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
Name
Status
Type
Initiator
Size
Time
36 / 315 requests
2,206 kB / 3,259 kB transferred
2,295 kB / 5,646 kB resources


oneramp.ammaammayi.in
ABOUT US
CONTACT US
OFFICE LOCATOR
DISCOVER
SETTINGS
Request For Quote
Perishables - AI
Product Name
Perishables
Product Type Code
Fresh Fruits and Vegetables
Product Type
Fruits & Vegetables
Special Code
PER
Booking Details
FRA
KBL
18 May 2026
20
600
K
ABCD
Consolidation
Special Codes
Add
Clear
PER
Dimensions
No dimensions added
Shipper Details
Select shipper code
Handling Information
Enter special service
Enter other service
Embargo
Continue


oneramp.ammaammayi.in
Quote Request Rate
18th may
Success
Request submitted successfully.
Enquiry Number
ENQ-2026-05-00701
OK
Original
Request
Weight Charge
USD
Weight Charge
USD 1200.00
Rate Request
No

Objective

To verify the API response performance and backend communication stability during the RFQ (Request For Quote) and enquiry creation workflow.

Modules Tested

- User Login
- RFQ V2 Workflow
- Quote Request Submission
- Enquiry Creation
- Quote List Loading

Testing Method

API response testing was performed using Chrome DevTools Network Monitoring.

The following workflow was executed manually:

1. User Login
2. RFQ Product Selection
3. Booking Detail Entry
4. Quote Request Submission
5. Enquiry Generation
6. Quote List Loading

Network tab → Fetch/XHR was used to monitor API requests, status codes, and response timings.

2.API Response Observations

API / Action	Status	Response Time	Result
Login API	200 OK	~285 ms	Successful
Customer Search API	200 OK	~939 ms	Successful
RFQ Create API	200 OK	~747 ms	Successful
Rates API	200 OK	~955 ms	Successful
Send Generic Email API	200 OK	~400 ms	Successful
Enquiry Queue Create API	200 OK	Successful response observed	Successful

---

Performance Findings

- APIs responded successfully with HTTP 200 status codes.
- Login API response was stable and fast.
- RFQ and enquiry creation completed successfully without failure.
- No API timeout or request failure was observed during execution.
- Backend communication between frontend and server was functioning correctly.

Bottlenecks Observed

- Multiple repeated background API calls were observed.
- Continuous business rule API calls may affect UI responsiveness.
- Heavy XHR activity may increase browser resource usage during long sessions.

Conclusion

API Response Time Testing was successfully completed for the RFQ and enquiry creation workflow.

The application APIs responded successfully with acceptable response timings during login, RFQ creation, quote request submission, and enquiry generation.

No critical API failures or timeout issues were identified during testing.

Scope Limitation

The following areas were not covered in this testing phase:

- Multi-user load testing
- Stress testing
- Concurrent user simulation
- Server scalability testing

3. Multiple User Load Testing

Tool Used

Grafana k6

Test Scripts Used

- drona_test.js
- signalr-test.js

The scripts simulated multiple users accessing the customer portal and SignalR communication endpoints.

Load Test Configuration

Parameter	Value
Virtual Users (VUs)	10
Test Duration	30 Seconds
Total Requests	283
Protocol	HTTPS

---

k6 Load Test Results

Metric	Result
Checks Succeeded	100%
Checks Failed	0%
Average Response Time	59.65 ms
Minimum Response Time	42.16 ms
Maximum Response Time	135.19 ms
HTTP Failures	0%
Iterations Completed	283

SignalR Realtime Testing

Results

Metric	Result
Realtime Connection	Successful
Status Code	200

Average Response Time	~59 ms
Response Received	Yes

Observations

- SignalR negotiation endpoint responded correctly.
- Realtime communication remained stable during concurrent execution.
- No websocket or negotiation failures were observed.

Overall Observations

- Application handled concurrent users successfully.
- APIs remained stable under load.
- No server crashes or request failures occurred.
- Response times were within acceptable range.
- Frontend optimization opportunities were identified through Lighthouse diagnostics.

Bottlenecks Identified

1. Render-blocking frontend resources
2. Large CSS and JavaScript payloads
3. Unused frontend assets
4. Large network transfer size
5. Initial page load delay
6. Potential backend load increase during extended realtime communication

---

Recommendations

1. Minify CSS and JavaScript files
2. Reduce unused frontend resources
3. Implement better browser caching

4. Optimize large assets and images
5. Improve frontend rendering performance
6. Monitor realtime SignalR traffic during high concurrency

Conclusion

Performance testing was successfully completed for the Drona Aviation Customer Portal using:

- Google Lighthouse
- Browser Network Monitoring
- Grafana k6 Load Testing

The application successfully handled concurrent users with stable API responses and zero request failures during testing.

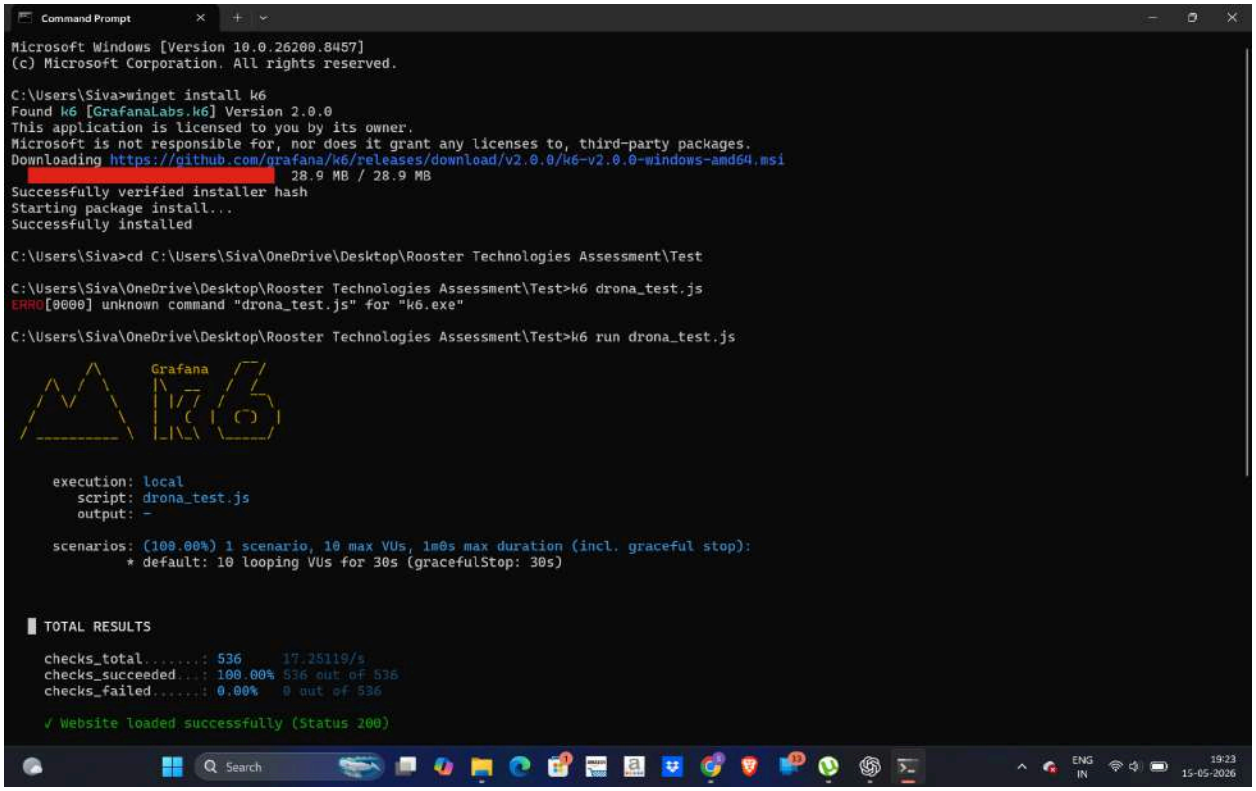
Frontend optimization improvements are recommended to enhance initial page loading speed and overall user experience.

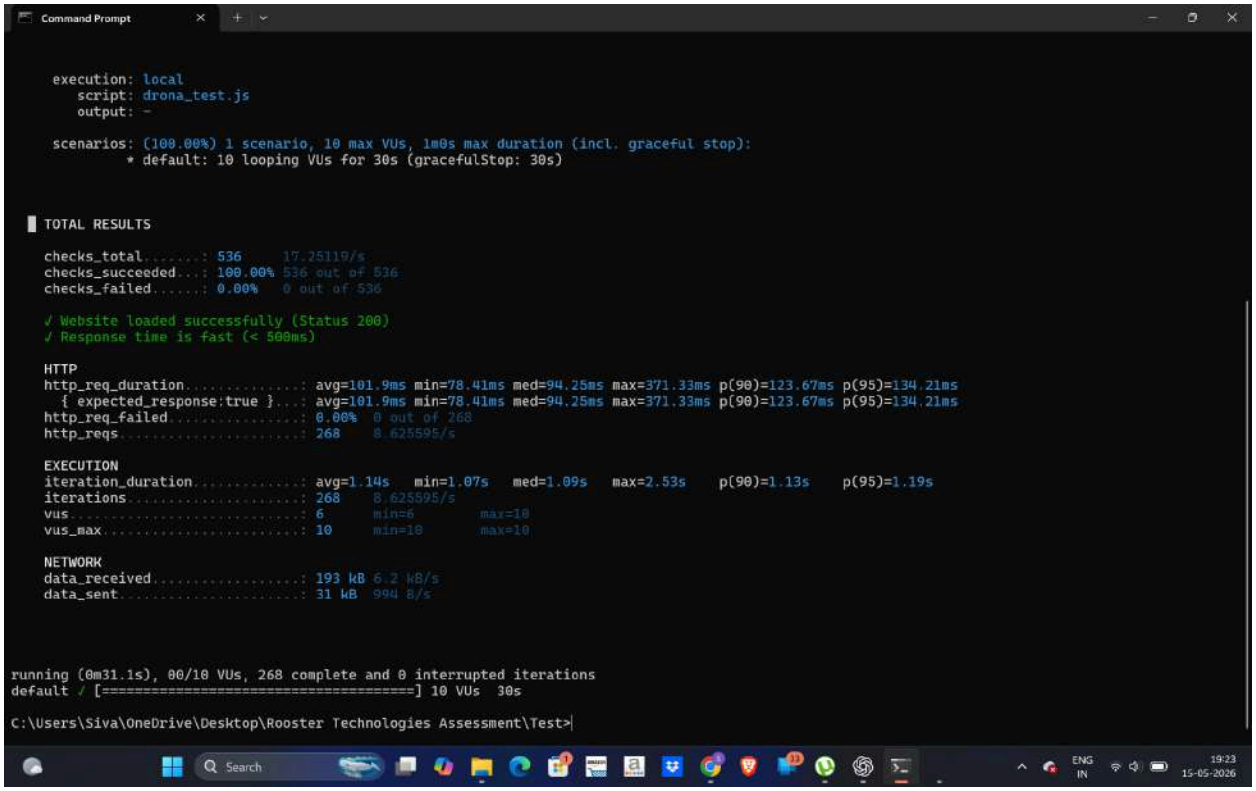
Customer Portal: https://oneramp.ammaammayi.in/

Javascript code:

```
import http from 'k6/http';

import { check, sleep } from 'k6';

// AI Generated Configuration: 10 Users for 30 Seconds
export const options = {
  vus: 10,
  duration: '30s',
};

// AI Generated Action: Hitting the Drona Aviation Customer Portal
export default function () {
  // Make a request to the Customer Portal URL from the QA document
  const res = http.get('https://oneramp.ammaammayi.in/');

  // AI checking if the site is working properly
  check(res, {
```

```
  'Website loaded successfully (Status 200)': (r) => r.status === 200,
  'Response time is fast (< 500ms)': (r) => r.timings.duration < 500,
 });

 // Short pause before the next request
 sleep(1);
}
```





Signalr-test

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 10,
  duration: '30s',
};

export default function () {

  let url =
'https://airline.drona-aviation.com/signalr/negotiate?clientProtocol=1.5&connectionData=%5B%7
B%22name%22%3A%22abpcommonhub%22%7D%5D';
```

```
  let headers = {
    'accept': 'text/plain, */*; q=0.01',
    'x-requested-with': 'XMLHttpRequest',
  };

  let response = http.get(url, { headers });

  check(response, {
    'status is 200': (r) => r.status === 200,
    'response received': (r) => r.body.length > 0,
  });

  console.log(`Response Time: ${response.timings.duration} ms`);

  sleep(1);
}
```

```
Command Prompt

C:\Users\Siva\OneDrive\Desktop\Rooster Technologies Assessment\Test>k6 run signalr-test.js

        /\      Grafana   /‾‾/
     /\  /  \     |\  __   /  /
    /  \/    \    | |/ /  /   ‾‾\
   /          \   |   (  |  (‾)  |
  / __________ \  |_|\_\  \_____/

     execution: local
        script: signalr-test.js
        output: -

     scenarios: (100.00%) 1 scenario, 10 max VUs, 1m0s max duration (incl. graceful stop):
              * default: 10 looping VUs for 30s (gracefulStop: 30s)

INFO[0000] Response Time: 49.7324 ms                    source=console
INFO[0000] Response Time: 54.046 ms                     source=console
INFO[0000] Response Time: 52.4217 ms                    source=console
INFO[0000] Response Time: 51.6853 ms                    source=console
INFO[0000] Response Time: 58.0597 ms                    source=console
INFO[0000] Response Time: 52.5056 ms                    source=console
INFO[0000] Response Time: 80.0243 ms                    source=console
INFO[0000] Response Time: 55.7404 ms                    source=console
INFO[0000] Response Time: 70.6596 ms                    source=console
INFO[0000] Response Time: 63.2236 ms                    source=console
INFO[0001] Response Time: 48.416 ms                     source=console
INFO[0001] Response Time: 57.8103 ms                    source=console
INFO[0001] Response Time: 60.3925 ms                    source=console
INFO[0001] Response Time: 55.5003 ms                    source=console
INFO[0001] Response Time: 53.734 ms                     source=console
INFO[0001] Response Time: 54.7294 ms                    source=console
INFO[0001] Response Time: 57.7331 ms                    source=console
INFO[0001] Response Time: 58.0234 ms                    source=console
INFO[0002] Response Time: 70.4297 ms                    source=console
INFO[0002] Response Time: 58.2313 ms                    source=console
INFO[0002] Response Time: 44.7577 ms                    source=console
INFO[0002] Response Time: 54.7097 ms                    source=console
INFO[0002] Response Time: 56.7557 ms                    source=console
INFO[0002] Response Time: 67.1191 ms                    source=console
INFO[0002] Response Time: 56.0873 ms                    source=console
INFO[0002] Response Time: 46.6711 ms                    source=console
```

```
Command Prompt

INFO[0029] Response Time: 52.6685 ms                    source=console
INFO[0029] Response Time: 53.9563 ms                    source=console
INFO[0029] Response Time: 54.899 ms                     source=console
INFO[0029] Response Time: 61.1243 ms                    source=console
INFO[0029] Response Time: 52.315 ms                     source=console
INFO[0029] Response Time: 58.0399 ms                    source=console
INFO[0029] Response Time: 58.7247 ms                    source=console
INFO[0030] Response Time: 47.8268 ms                    source=console
INFO[0030] Response Time: 47.2425 ms                    source=console
INFO[0030] Response Time: 53.5147 ms                    source=console

  TOTAL RESULTS

    checks_total.......: 566     18.249018/s
    checks_succeeded...: 100.00% 566 out of 566
    checks_failed......: 0.00%   0 out of 566

    ✓ status is 200
    ✓ response received

    HTTP
    http_req_duration..............: avg=59.65ms min=42.16ms med=56.56ms max=135.19ms p(90)=72.67ms p(95)=80.71ms
      { expected_response:true }...: avg=59.65ms min=42.16ms med=56.56ms max=135.19ms p(90)=72.67ms p(95)=80.71ms
    http_req_failed................: 0.00%  0 out of 283
    http_reqs......................: 283    9.124509/s

    EXECUTION
    iteration_duration.............: avg=1.08s   min=1.04s   med=1.05s   max=1.86s   p(90)=1.07s   p(95)=1.11s
    iterations.....................: 283    9.124509/s
    vus............................: 3      min=3        max=10
    vus_max........................: 10     min=10       max=10

    NETWORK
    data_received..................: 204 kB 6.6 kB/s
    data_sent......................: 30 kB  976 B/s


running (0m31.0s), 00/10 VUs, 283 complete and 0 interrupted iterations
default ✓ [======================================] 10 VUs  30s
```

==========

4.Bottleneck Identification Report

Module

Drona Aviation Customer Portal

Objective

The objective of this testing was to identify performance bottlenecks affecting application speed, frontend rendering, API efficiency, and overall user experience.

Tools Used

1. Google Lighthouse
2. Chrome DevTools Network Monitoring
3. Grafana k6 Load Testing

Bottlenecks Identified

1. Slow Initial Page Rendering

Observation

The application took noticeable time during initial page loading.

Evidence

Metric	Result
First Contentful Paint	5.2 seconds

Speed Index	9.0 seconds

Impact

- Users experience slow initial screen rendering.
- Delayed visibility of important UI elements.
- Reduced user experience quality.

---

2. Render Blocking Resources

Observation

Multiple frontend resources blocked page rendering during startup.

Lighthouse Result

- Render-blocking requests detected.

Impact

- Delayed UI rendering.
- Increased page load time.
- Browser waits for CSS/JS execution before displaying content.

---

3. Unoptimized CSS Files

Observation

Unused and non-minified CSS resources were detected.

Diagnostics

- Minify CSS — Error
- Reduce unused CSS — Error

Impact

- Increased frontend payload size.
- Slower page rendering.
- Higher browser processing time.

---

4. Unoptimized JavaScript

Observation

Large and unused JavaScript resources were identified.

Diagnostics

- Minify JavaScript — Error
- Reduce unused JavaScript — Error
- Legacy JavaScript detected

Impact

- Increased CPU usage.
- Slower frontend execution.
- Higher blocking time during page rendering.

---

5. Large Network Payloads

Observation

Large frontend resource transfer size detected.

Lighthouse Insight

- Avoid enormous network payloads.

Impact

- Slower loading for users with low internet speed.

- Increased bandwidth usage.
- Delayed application responsiveness.

---

6. Cache Optimization Issues

Observation

Browser caching strategy was not fully optimized.

Lighthouse Insight

- Use efficient cache lifetimes.

Estimated Savings

- Approximate savings: 5,137 KiB

Impact

- Repeated resource downloads.
- Increased load time during repeated visits.
- Higher network consumption.

---

7. Backend API Load Under Concurrent Users

Observation

Multiple API calls were triggered during concurrent operations.

Test Result

- APIs responded successfully with Status 200.
- No failures observed during current load test.

Potential Risk

Under higher user load, backend services may experience:

- Increased response time
- Queue delays
- Database load increase

---

8. Realtime SignalR Communication Load

Observation

SignalR realtime negotiation APIs handled requests successfully.

Test Result

Metric	Result
Average Response Time	~59 ms
Failures	0%

Potential Risk

High concurrent realtime traffic may increase:

- WebSocket server load
- Memory consumption
- Persistent connection overhead

Bottleneck Severity Summary

Bottleneck	Severity
Slow Initial Rendering	High
Render Blocking Resources	High
Unoptimized JavaScript	High
Unoptimized CSS	Medium
Large Payload Size	Medium
Cache Optimization Issues	Medium
Backend Concurrent Load	Medium
SignalR Realtime Load	Medium

Recommendations

Frontend Optimization

- Minify CSS and JavaScript
- Remove unused resources
- Optimize bundle size

- Implement lazy loading

Caching Improvements

- Configure browser cache headers
- Enable long-term static asset caching

Backend Improvements

- Monitor API response times under heavy load
- Optimize database queries
- Implement API caching where possible

Realtime Communication Improvements

- Monitor SignalR concurrent connections
- Optimize websocket session handling

Conclusion

The performance bottleneck analysis identified several frontend optimization opportunities, especially related to rendering performance, unused resources, caching, and large network payloads.

Despite these bottlenecks, the application remained stable during concurrent load testing and API testing, with zero request failures observed.

==========